

现有 YASA 技术路线调研报告

为更好地理解课题，团队调研并整理了现有 yasa 的技术实现，主要针对 yasa-uast 和 yasa-engine 模块进行研究分析。

YASA 是一个开源的静态程序分析项目。它的核心创新之处在于采用了一种名为 UAST 的统一中间表示形式，这种设计旨在支持多种编程语言。在 UAST 的基础上，YASA 提供了一个非常精确的静态分析框架。

1 UAST 统一中间表示

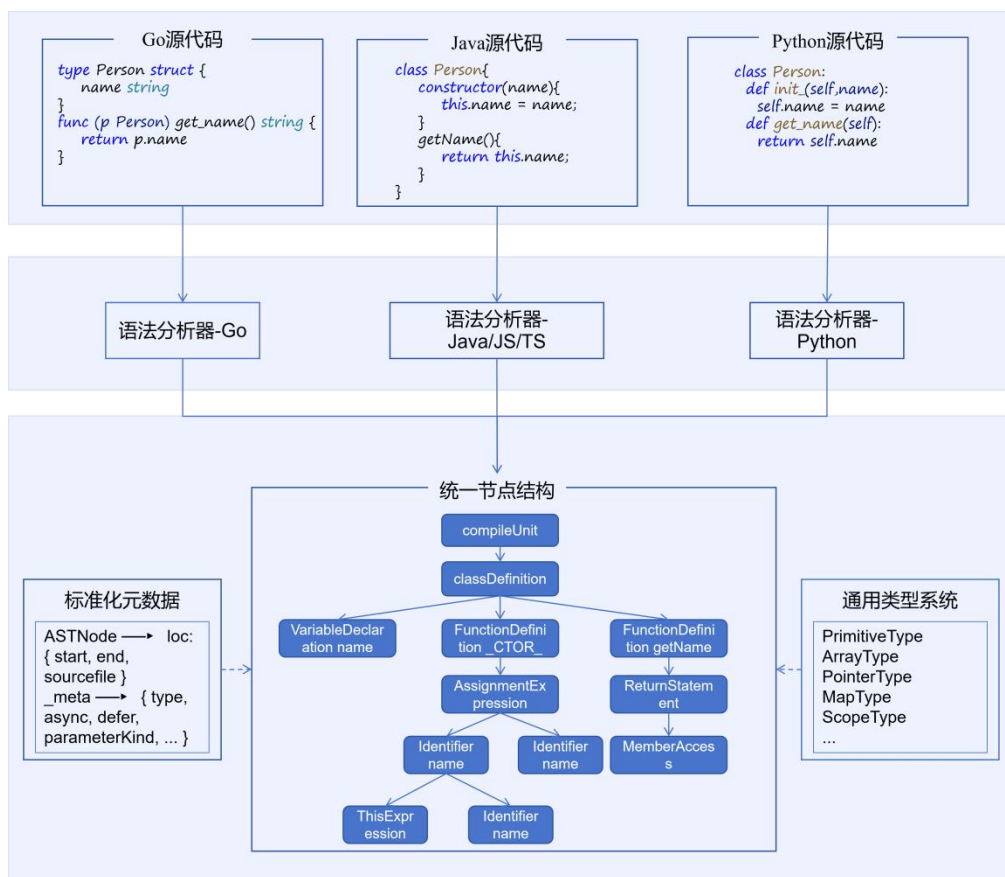


图 1 UAST 流程图

YASA-UAST 的主要目的是将来自不同编程语言的源代码标准化，转换为一种统一的树形结构。这种标准化使得后续的分析引擎能够执行诸如污染分析、数据流分析以及函数调用图生成等任务，而无需为每个检查器实现特定于语言的逻辑。

分别输入 Go、Java、Python 等不同语言的源代码，各语言代码先经过对应的语法分析器解析，随后被转换并汇聚到统一的节点结构中。该统一结构以 `compileUnit` 为入口，向下组织类定义、变量声明、函数定义、赋值表达式、返回语句、标识符、成员访问、`this` 表达式等通用语法节点，从而屏蔽不同编程语言在语法形式上的差异。图中左侧展示了节点的标准元数据，例如位置信息、源码文件信息及节点附加属性；右侧展示了通用类型系统，包括基础类型、数组类型、指针类型、映射类型、作用域类型等。

UAST 的转换本质是一种有损统一抽象，为用尽可能简单的语法信息做统一表达，与程序分析无关的信息会被去除，只保留分析需要的信息与语言特性。

2 项目级上下文构造

YASA-Engine 是建立在 UAST 之上的统一语义分析执行层，负责把标准化语法结构进一步转化为可用于调用图、数据流、污点传播和规则检测的程序分析事实。

YASA-Engine 在 UAST 统一中间表示基础上,进一步构建了面向项目级静态分析的语义基础。UAST 能够统一不同编程语言的语法结构,但真实软件项目并不是由孤立语法节点组成,而是由源码文件、目录结构、依赖声明、框架配置、模块关系、语言语义和框架约定共同构成。对于多语言、多框架程序而言,仅依赖单个文件的语法树,难以判断程序入口在哪里、标识符绑定到哪个实体、函数调用是否跨文件可达、依赖包中的 API 是否参与安全敏感路径,也难以识别框架驱动产生的隐式调用关系。

因此, YASA-Engine 在 UAST 与核心静态分析框架之间设置了项目级语义构造机制。该机制的作用不是直接执行漏洞检测,而是将上游输入的 UAST、源码文件、配置文件、依赖声明、目录结构和框架约定等分散工程信息,组织为核心分析框架可以消费的项目级语义对象。通过这一过程, YASA-Engine 能够从单文件语法分析扩展到项目级程序分析,为后续调用图构建、数据流分析、跨过程分析、污点传播和安全检测提供基础。

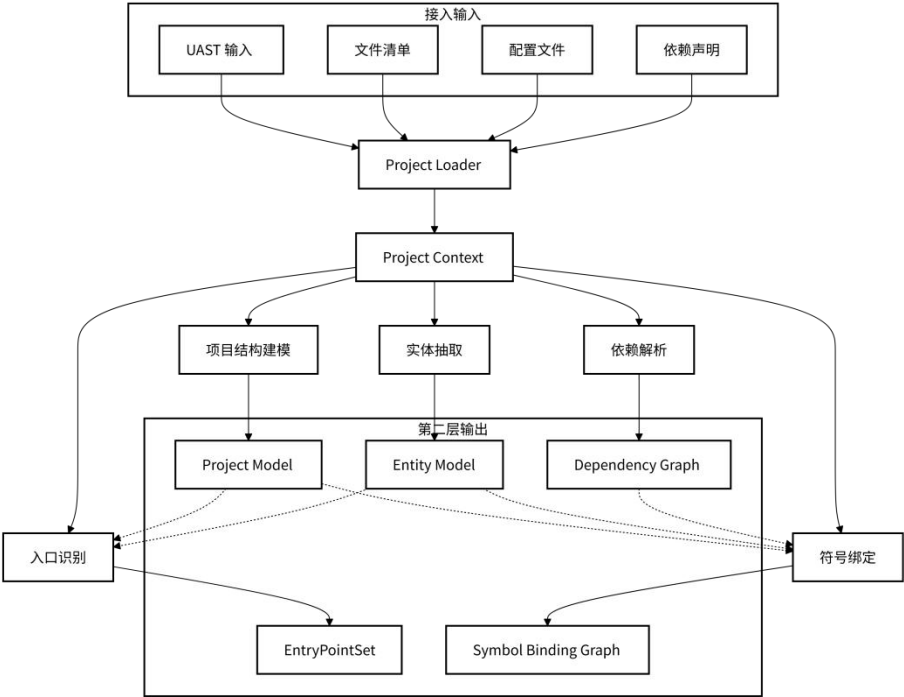


图 2 YASA-Engine 上下文构建流程

(1) 项目上下文构建：从 UAST 到 Project Context

YASA-Engine 的项目级语义构造以 Project Loader 为入口。Project Loader 负责接入 UAST 输入以及文件清单、配置文件、依赖声明等工程元信息,并对这些异构输入进行统一组织。其核心任务是将已有 UAST 与项目文件、源码位置、目录结构、配置项和依赖信息建立关联,形成后续模块可共享的项目上下文 Project Context。

Project Context 可以理解为项目级语义构造过程中的基础工作空间。它并不是单一语法树,而是围绕完整工程建立的上下文容器,用于支撑项目结构建模、实体抽取、入口点识别、依赖解析、符号绑定以及后续语言和框架语义补充等步骤。

在 Project Loader 建立项目上下文之后, YASA-Engine 的分析对象从孤立的源码文件扩展为完整工程。这样,后续分析可以在同一项目上下文中理解文件之间的引用关系、模块之间的依赖关系、配置与代码之间的关联关系,以及框架约定对程序入口和调用关系的影响。

(2) Project Model 与 Entity Model 构建

在项目上下文基础上, YASA-Engine 首先形成 Project Model。Project Model 用于描述项目的文件组织、目录结构、模块边界、语言类型以及与配置、依赖相关的工程信息。通过 Project Model, YASA-Engine 能够以工程为单位进行分析,而不是以孤立文件为单位进行扫描,从而为后续跨文件、跨模块和跨依赖分析提供上下文基础。

系统需要从 UAST 及项目上下文中抽取程序实体,形成 Entity Model。Entity Model 用于描述函数、类、方法、变量、参数、返回值、模块以及外部 API 等程序实体,将 UAST

中的语法节点进一步整理为安全分析可使用的程序实体对象。UAST 能够表示语法结构，但后续调用图构建、数据流分析和污点传播分析需要处理的是函数、方法、参数、对象成员和外部调用等语义实体，因此实体抽取是从语法表示走向语义分析的必要环节。

Project Model 和 Entity Model 共同构成项目级语义构造的基础。后续入口识别、依赖解析、符号绑定以及语言和框架适配都依赖这两类模型。

(3) 入口点识别机制

EntryPointCollector 负责在项目上下文和实体模型基础上识别程序入口点，形成 EntryPointSet。其核心思想是将入口点识别从传统 main 函数扩展到框架驱动入口。

在传统程序中，入口可能表现为 main 函数或命令行入口；而在 Web 和服务端框架中，入口则可能表现为 Spring Controller 方法、Flask/FastAPI 路由装饰器、Gin Handler 注册、Egg 控制器约定、Node Web 路由处理函数或 MCP SDK 工具处理函数。EntryPointCollector 通过分析 UAST 结构、配置线索、框架特征、注解/装饰器、路由注册语句和实体信息，收集可能触发程序执行的入口点。

EntryPointSet 不仅记录入口函数的位置，理想情况下还应标注入口类型、触发机制、框架来源、参数来源以及路由或事件语义。例如，对于 Web 路由入口，入口点信息不仅应包括处理函数，还应包括 HTTP 方法、路由路径、请求参数来源和上下文对象。对于事件回调或 SDK 工具处理函数，则应记录触发方式和参数绑定关系。

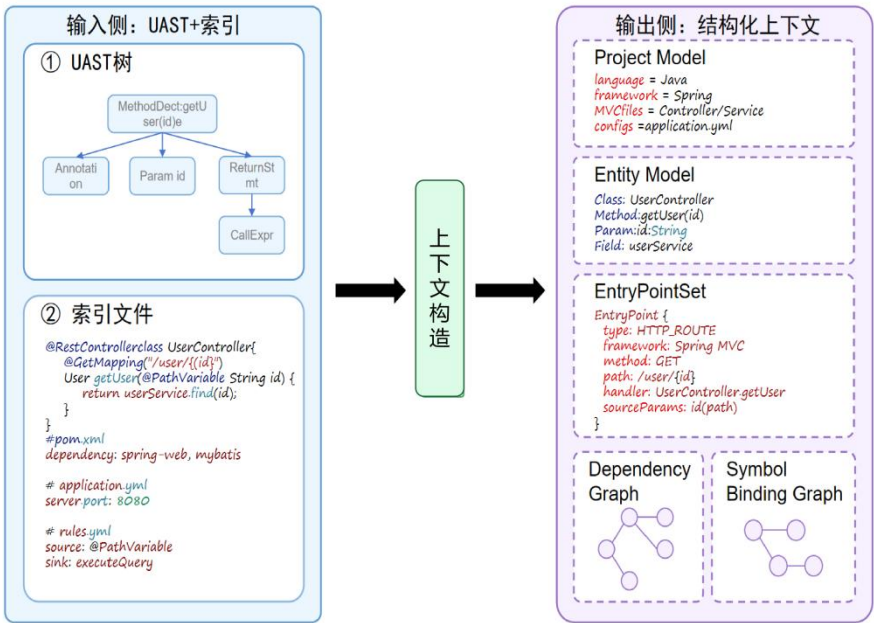


图 3 上下文构造生成示例

(4) 依赖解析与模块关系建模

依赖解析模块负责处理项目内部模块以及外部依赖之间的关系，形成 Dependency Graph。该模块通过语言导入语句、依赖声明、包管理信息、构建配置和项目目录结构，刻画文件、模块、包和第三方依赖之间的引用关系。

Dependency Graph 的作用是明确分析边界和跨模块可达关系，使后续分析能够从单文件扩展到项目级范围，而不是局限于局部函数或局部文件。安全漏洞常常跨文件、跨模块传播，例如控制器接收用户输入后调用服务层，服务层调用 DAO 层或 SDK，最终到达数据库查询、命令执行或网络请求等危险操作。若缺少依赖关系和模块边界信息，分析引擎难以恢复完整调用链和数据流路径。

在多语言、多框架项目中，依赖解析不仅需要处理项目内部模块，还需要识别第三方库、框架组件、SDK 包和外部 API 的引用关系。对于安全分析而言，依赖图不仅回答模块之间是否存在引用，还影响后续 Source、Sink、Sanitizer 规则能否正确映射到具体 API 或封装函数上。

(5) 符号解析与绑定机制

Resolver 负责完成符号解析与绑定，形成 Symbol Binding Graph。其作用不是简单查找

变量名，而是将 UAST 中出现的标识符、调用表达式、成员访问和模块引用绑定到具体程序实体。

Resolver 需要结合 Project Model、Entity Model 和 Dependency Graph，处理变量、函数、类、方法、模块、包和依赖之间的引用关系。通过符号绑定，YASA-Engine 才能判断某个调用表达式实际指向哪个函数或方法，某个成员访问对应哪个对象或字段，某个模块引用来自项目内部还是外部依赖。

Symbol Binding Graph 解决语法节点指向哪个程序实体的问题，是调用图构建和跨过程数据流分析的重要依据。调用图需要知道某个调用表达式实际指向哪个函数、方法或外部 API；数据流分析需要知道变量、参数、返回值和字段之间的真实绑定关系；污点传播分析则需要在这些绑定关系基础上判断外部输入是否能够跨函数、跨对象或跨模块传播到危险操作点。

3 多语言适配

在项目上下文、实体模型、入口点、依赖关系和符号绑定等基础语义信息形成之后，YASA-Engine 仍需要进一步处理多语言、多框架和多类 SDK/API 带来的语义差异。UAST 能够在一定程度上统一不同编程语言的语法结构，但无法直接消除不同语言在类型系统、模块机制、对象模型、调用分派、运行时约定以及框架调度机制上的差异。YASA-Engine 需要通过多语言语义适配、框架语义建模、SDK/API 安全语义补充和规则映射机制，将不同语言和框架中的特定语义规约为核心分析层和安全检测层可复用的统一结构化信息。

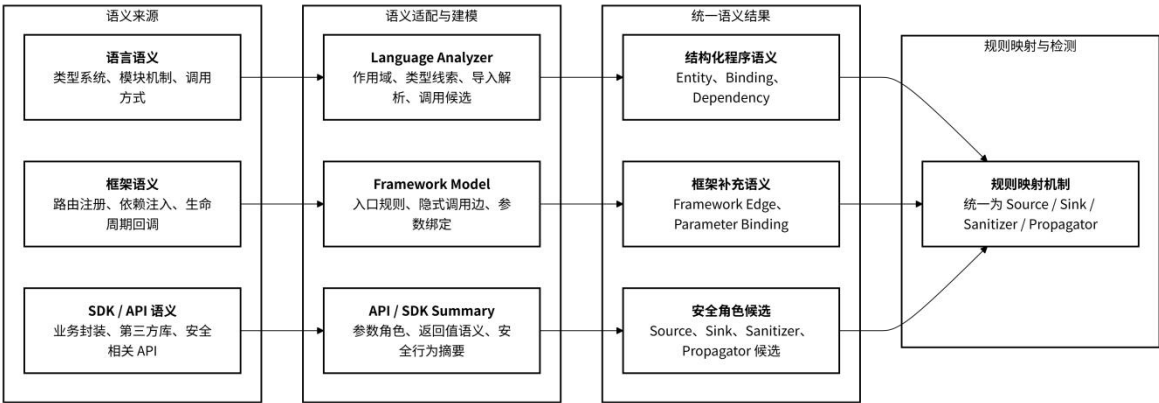


图 4 多语言多框架语义适配与安全规则映射机制图

(1) 多语言语义适配机制

多语言语义适配机制主要面向不同编程语言的基础语义差异。不同语言在函数、类、方法、变量、模块、包、作用域、类型系统和调用机制等方面具有不同的表达方式和解析规则。语言 Analyzer 的作用并不是重新构建一套独立于项目级语义模型之外的分析产物，而是作为语言特定语义的适配单元，将不同语言中的特殊语义解释为可被统一 Entity Model、Symbol Binding Graph、Dependency Graph 以及调用图候选关系复用的结构化信息。

具体而言，语言 Analyzer 需要根据目标语言的语法和语义规则，对程序实体、作用域关系、模块导入、成员访问、类型线索和调用候选进行解析，并将解析结果映射到统一语义模型中。例如，Java 语言中的类层次、接口派发、注解、方法重载和依赖注入会影响实体识别、符号绑定和调用目标解析；Python 语言中的装饰器、动态导入、模块作用域和鸭子类型会影响入口识别、模块解析和调用候选推断；JavaScript/TypeScript 中的异步回调、闭包、CommonJS/ESM 模块和原型链会影响控制流恢复、作用域分析和对象成员绑定；Go 语言中的包机制、接口方法和接收者函数会影响包级依赖、方法归属和接口调用解析；PHP 中的命名空间、include/autoload 和动态函数调用则会影响跨文件引用和动态调用解析。通过语言 Analyzer 的适配，YASA-Engine 能够将不同语言中的基础语义差异统一为后续核心分析可使用的实体、绑定、依赖和调用候选信息。

(2) 框架语义建模机制

除语言层面的差异外，现代应用程序还大量依赖 Web 框架、服务端框架和 SDK 框架的运行时约定。许多程序入口、调用关系、参数来源和对象绑定关系并不直接表现为源码中的显式函数调用，而是由注解、装饰器、配置文件、路由注册、依赖注入、生命周期回调、事件监听、中间件链或协议回调隐式决定。框架语义建模机制的作用在于补充语言本身无法表达的框架运行时语义，将框架约定转化为静态分析可使用的结构化语义信息。

对于 Web 框架和服务端框架，框架语义模型需要描述入口识别规则、路由注册方式、参数绑定规则、依赖注入方式、生命周期回调、事件监听、上下文对象和隐式调用边。例如，Spring 框架语义模型需要识别 Controller 注解、RequestMapping 系列注解、Bean 注入和请求参数绑定关系；FastAPI 框架语义模型需要识别 app.get、router.post、Depends 以及请求参数来源；Gin 框架语义模型需要识别 GET、POST、Group 等路由注册方式以及 Context 参数读取方式；Node/Egg 类框架则需要结合目录约定、路由配置和控制器方法恢复入口关系。

框架语义建模的关键产物包括 Framework Edge 和 Parameter Binding Model。Framework Edge 用于描述源码中不显式出现、但由框架运行时调度产生的控制流关系，例如从框架路由分发器到用户 Handler 的调用边、从依赖注入容器到具体 Bean 的绑定边、从事件注册点到回调函数的触发边等。这些关系可以作为调用图构建过程中的隐式调用边补充。Parameter Binding Model 用于描述请求参数、配置值、上下文对象字段和函数参数之间的绑定关系，例如 HTTP path 参数如何绑定到 Controller 方法参数，请求体字段如何绑定到对象属性，Context 对象中的 query 参数如何传播到局部变量。该类模型直接影响污点源识别、数据流传播和跨过程路径恢复的精度。

(3) SDK 与 API 安全语义补充

在真实工程项目中，许多 Source、Sink、Sanitizer 和 Propagator 并不直接出现为标准库或框架 API 调用，而是被业务 SDK、数据库 SDK、云服务 SDK 或内部工具函数封装。项目级语义构造能力仅识别函数调用形式仍然不足以支撑准确的安全检测，还需要为安全相关 SDK 和 API 建立语义摘要，描述其参数角色、返回值语义、数据传播行为和安全作用。

API Semantic Summary 用于描述特定 API 的安全语义摘要，包括参数角色、返回值含义、是否读取外部输入、是否执行危险操作、是否传播污点、是否具有净化效果以及适用漏洞类型等信息。该结构可以支撑 Source、Sink、Sanitizer 和 Propagator 规则扩展，也可以作为后续大语言模型语义增强的重要结构化输入。SDK Behavior Model 用于描述 SDK 调用的抽象行为，包括是否访问网络、数据库、文件系统、命令执行环境、外部服务或敏感资源，是否进行编码、过滤、校验或权限检查，以及是否可能引入外部输入或危险输出。该模型有助于提升 YASA-Engine 对第三方库、业务封装和跨服务调用的理解能力。

(4) 规则映射机制

规则映射机制用于将语言语义、框架语义和 SDK/API 语义进一步转换为 Checker 可直接使用的安全规则输入。不同语言和框架中可能存在语义等价但形式不同的 API 或调用模式，规则映射机制需要将这些差异化表达统一到 Source、Sink、Sanitizer 和 Propagator 等安全语义类别下，使 Checker 层能够基于统一规则模型开展检测，而不必直接处理所有语言、框架和 SDK 的细节差异。

例如，不同框架中的请求参数读取方法可以统一映射为外部输入 Source；不同数据库执行 API、命令执行 API、文件访问 API 或网络请求 API 可以根据漏洞类型映射为相应 Sink；不同编码、转义、过滤、白名单校验和参数化查询函数则需要结合漏洞类型、参数位置、返回值语义和上下文条件映射为特定 Sanitizer；对象赋值、参数传递、返回值传递和封装函数则可以根据数据传播行为映射为 Propagator。通过这种规则映射机制，Checker 层能够围绕统一的安全语义类别执行规则匹配、污点传播、调用链分析和结果生成

4 静态分析执行

在完成项目级语义基础构造之后，YASA-Engine 的静态分析执行层以上游已经形成的项目级语义产物为输入，在此基础上继续恢复调用关系、数据流关系、指向关系和污点传播路径，将前述收集到的信息进一步转化为可被 Checker 消费的程序分析事实。以

BaseAnalyzer 为中心，围绕其组织 AnalysisContext、MemSpace、MemState、Scope Hierarchy、Symbol Table Manager 和 Result Manager 等组件，并在内部提供调用图构建、数据流分析、指向分析、值传播分析、跨过程分析和污点传播分析等基础能力。

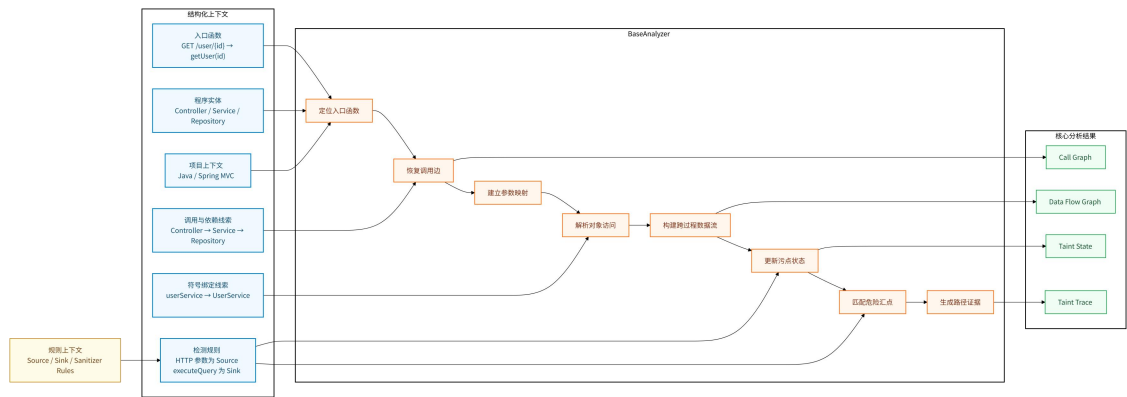


图 5 静态分析漏洞路径恢复流程图

(1) 构建核心分析上下文。
该上下文包含 UAST 节点，前一层输出的 Project Model、Entity Model、EntryPointSet、Dependency Graph 和 Symbol Binding Graph。Project Model 用于限定分析对象的工程范围、语言类型、目录结构和模块边界；Entity Model 提供函数、类、方法、变量、参数、模块和外部 API 等可引用实体；EntryPointSet 提供调用图和污点分析的起点；Dependency Graph 用于支持跨文件、跨模块和跨依赖传播；Symbol Binding Graph 则为调用表达式、标识符和成员访问提供初始绑定结果。在这些项目级信息的基础上，将 UAST 中的语法节点转化为可传播、可查询、可追踪的分析状态。

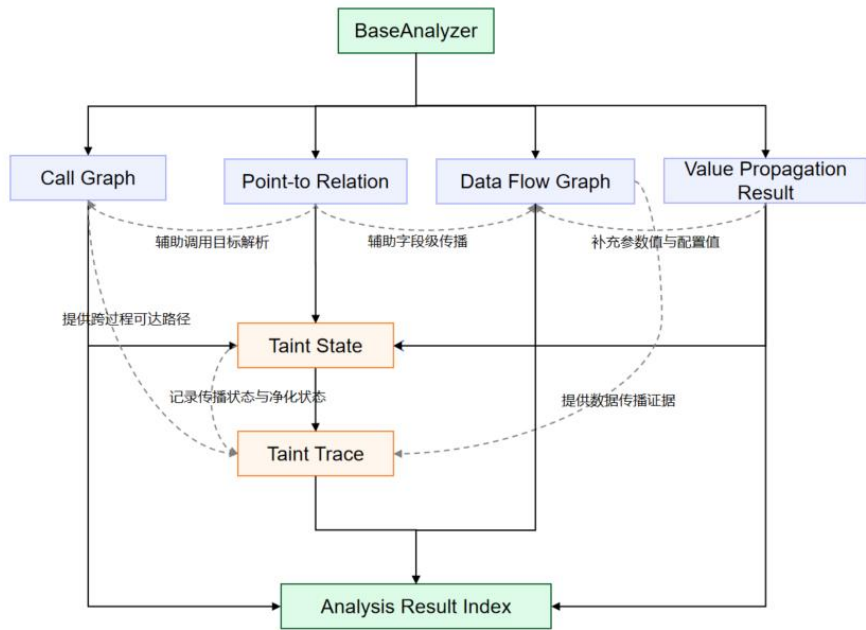


图 6 结构化分析结果

(2) 基于实体和绑定关系恢复调用目标。
根据 Symbol Binding Graph 在具体分析过程中结合指向关系、作用域状态和语言语义进行补充。对于普通函数调用，调用目标可以主要依赖已有符号绑定结果；对于跨文件调用，需要结合 Dependency Graph 判断被调用实体所在模块或依赖包；对于对象方法调用，需要结合接收者对象、类成员关系和指向分析结果推断候选方法；对于框架回调、路由函数、生命周期函数或 SDK 封装调用，则需要结合 EntryPointSet、Framework Profile 和 API Summary 等框架语义信息补充隐式调用关系。

(3) 指向分析与抽象状态传播。

根据 Entity Model 中的实体定义、Symbol Binding Graph 中的初始绑定关系，以及 Project Model 和 Dependency Graph 提供的跨模块上下文近似恢复变量、对象、字段、容器元素和调用接收者可能指向的抽象实体。指向分析尝试判断某个变量、对象引用或成员访问可能指向哪些抽象对象或程序实体，从而帮助解析动态方法调用、对象字段传播和多态派发。值传播分析则用于追踪常量、字符串、配置值、路径片段和参数值等信息，为路由识别、危险 API 参数判断和规则匹配提供补充依据。最终形成 Points-to Relation 和 Abstract State，为调用图精化、对象关系恢复和数据流传播提供依据。

(4) 调用图构建。

根据入口点、符号绑定关系、依赖关系和语言/框架规则，恢复函数、方法、回调和框架入口之间的调用关系。对于普通函数调用，调用图可以依据符号解析结果直接建立调用边；对于对象方法调用，需要结合类型信息、指向分析或候选绑定进行推断；对于框架驱动调用，则需要依赖 EntryPointSet 和框架规则补充隐式调用边。

(5) 数据流分析。

数据流分析在调用图和实体模型基础上，恢复变量、参数、返回值、对象字段和表达式之间的数据传递关系。不仅关注语句内部的数据赋值关系，还需要处理函数调用中的实参与形参映射、返回值传播、字段读写、集合元素传播以及跨过程数据传递。

(6) 污点传播与安全语义判断。

污点分析将 Source 规则、Sink 规则、Sanitizer 规则和 Propagator 规则映射到程序路径中，判断不可信输入是否能够经过若干传播步骤到达危险操作点。如果污点路径中存在有效净化节点，则需要根据净化规则调整污点状态；如果污点未经有效净化到达 Sink，则生成相应风险路径。

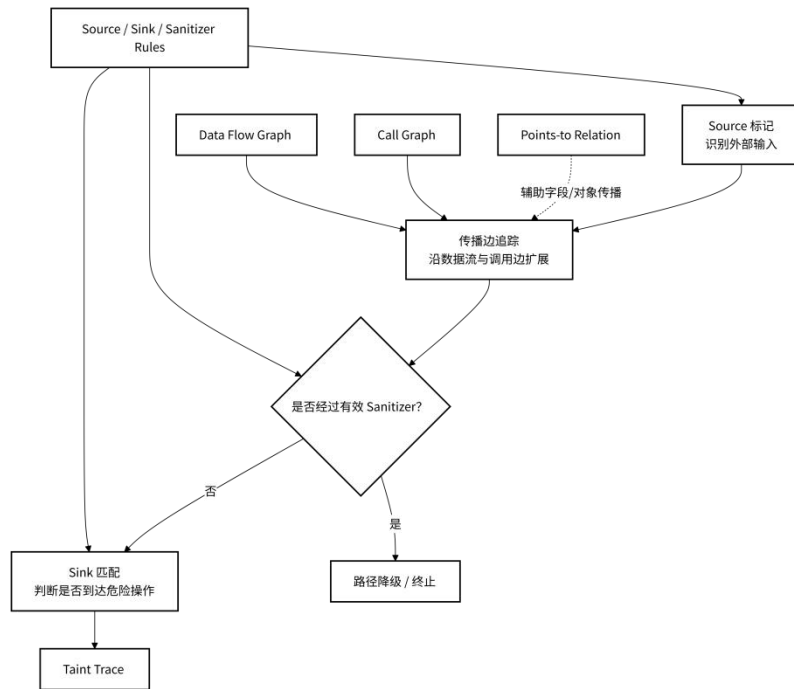


图 7 污点路径构造流程图

5 Checker 与结果输出

Checker 与结果输出将通用静态分析能力转化为可扩展的安全检测能力，并将检测结果以统一方式保存和输出。

现有 YASA-Engine 的 Checker 机制采用事件驱动和插件化设计。每个检测任务被封装为独立 Checker，不同 Checker 可以面向不同分析目标实现检测逻辑，例如污点路径检

测、调用图分析、调用链分析、Sanitizer 识别、AntiQL/UQL 查询或 SDK 语义检测。**CheckerManager** 负责 **Checker** 的注册、调度和生命周期管理，使不同检测任务能够以相对解耦的方式嵌入统一分析流程中。通过该机制，**YASA-Engine** 不需要将所有漏洞规则和检测逻辑硬编码在核心分析器内部，而是将底层分析能力与上层检测逻辑分离，提高了检测能力的扩展性和组合性。

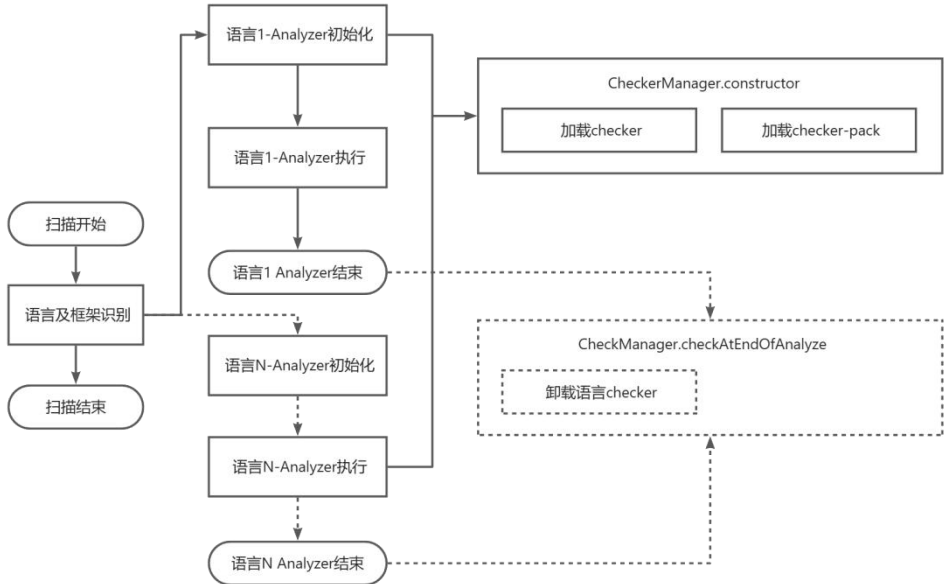


图 8 Checker 事件驱动调度机制示意图

（1）注册机制

Checker 可以通过配置文件自动注册，也可以在代码中手动注册。系统初始化时，**CheckerManager** 会加载待启用的 **Checker**，并根据 **Checker** 类中实现的处理方法，将其自动绑定到对应触发点。例如，若某个 **Checker** 实现了函数调用前处理方法，则该 **Checker** 会被注册到函数调用前触发点；若其实现了成员访问、赋值、条件判断或分析结束处理方法，则分别绑定到相应 **checkpoint**。该机制使 **Checker** 的能力边界由其实现的事件处理方法决定，新增检测能力时只需实现对应 **Checker** 及其触发点逻辑，不需要修改核心分析流程。

（2）执行过程

语言 **Analyzer** 会在特定分析时机触发事件，**CheckerManager** 根据当前事件类型获取已经注册的 **Checker** 列表，并依次调用对应处理方法。典型触发点包括分析开始、分析结束、文件分析前后、函数定义、函数调用前、函数调用后、变量声明、成员访问、赋值操作和条件判断等。在触发过程中，**Checker** 可以获得 **analyzer**、**scope**、**node**、**state**、**info** 等上下文信息。其中，**analyzer** 提供对全局分析状态和基础分析能力的访问，**scope** 表示当前作用域，**node** 表示当前 **UAST** 节点，**state** 表示当前执行状态或变量状态，**info** 则用于传递函数调用信息、参数信息或其他额外分析上下文。通过这些上下文信息，**Checker** 能够在关键程序点访问分析事实并执行检测逻辑。

（3）检测逻辑

不同 **Checker** 可以复用核心分析层已经形成的结构化结果。**TaintChecker** 主要消费 **Source/Sink/Sanitizer** 规则、数据流图、污点状态和污点传播路径，用于判断不可信输入是否能够到达危险操作点；**CallGraphAnalyzer** 主要消费实体关系、符号绑定和调用目标恢复结果，用于构建或输出函数调用关系；**CallChainChecker** 关注从入口点到目标函数或敏感 **API** 的调用链；**SanitizerChecker** 关注污点路径中的净化函数及其有效性；**AntiQL/UQL Checker** 面向查询规则执行模式匹配；**SDK Checker** 则用于处理特定 **SDK** 或外部 **API** 的语义规则。上述 **Checker** 虽然检测目标不同，但均建立在 **BaseAnalyzer** 产生的调用图、数据流图、指向关系、符号表、污点状态和规则配置之上。

关键特征在于，安全检测并非仅在分析结束后统一扫描一次，而是可以嵌入核心分析过

程，在特定事件点实时执行。例如，在函数调用前触发点，Checker 可以检查当前调用是否命中 Sink 规则，并结合参数污点状态判断是否形成风险；在成员访问触发点，Checker 可以判断对象属性是否涉及框架输入对象或敏感字段；在赋值触发点，Checker 可以更新传播关系或记录变量状态；在分析结束触发点，Checker 可以汇总路径、去重结果、清理资源或生成最终输出。通过事件驱动机制，Checker 能够与静态分析过程保持同步，使检测逻辑与程序语义恢复过程紧密结合。

(4) 规则约束

Source/Sink/Sanitizer 规则、Checker Packs 和查询规则通过配置加载进入系统后，会在相应 Checker 中被消费。Source 规则用于确定不可信输入来源，Sink 规则用于确定安全敏感操作点，Sanitizer 规则用于判断路径中是否存在有效净化逻辑，Checker Packs 用于组织不同语言、框架和漏洞类型的检测任务，查询规则则为 UQL 或 AntiQL 类 Checker 提供模式匹配依据。由此，Checker 层将前序规则映射机制形成的安全语义规则落到具体分析事件和检测任务中，实现从规则配置到漏洞结果的转换。

(5) 结果保存

YASA-Engine 将检测逻辑与输出逻辑分离，Checker 在检测到风险或生成分析结果时，并不直接负责具体文件输出，而是构造 Finding 并写入 ResultManager。ResultManager 作为统一结果管理组件，用于集中保存不同 Checker 产生的结果，并以输出策略标识对结果进行分类组织。该设计使 Checker 可以专注于检测逻辑本身，而不需要关心结果最终以何种格式、何种文件路径、何种序列化方式输出。

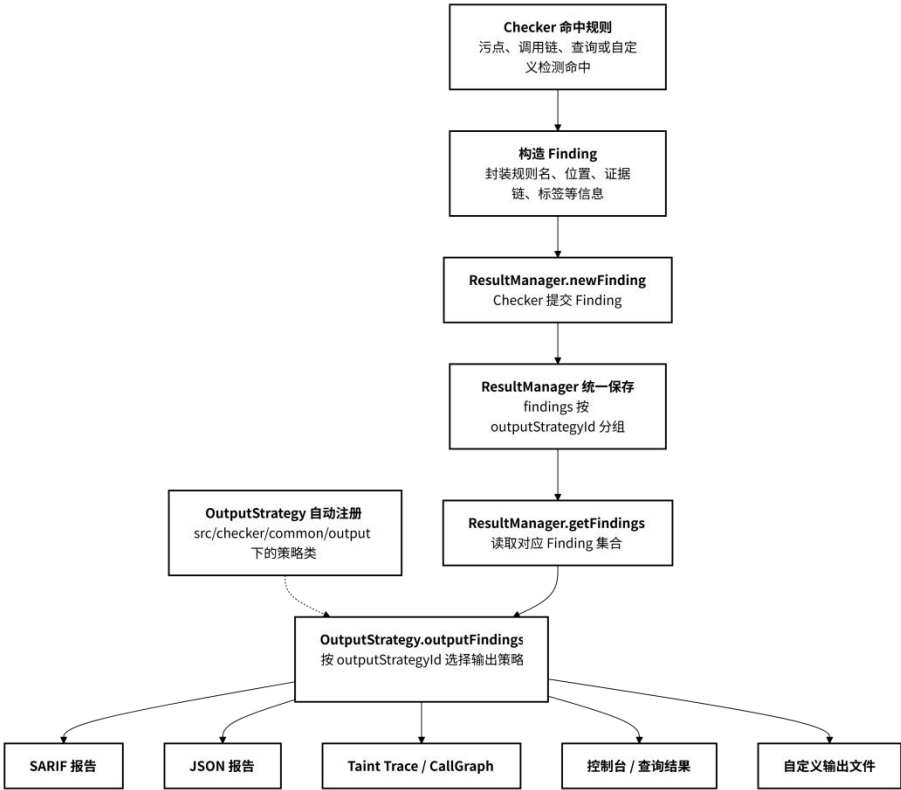


图 9 Finding 结果管理与输出策略图

(6) OutputStrategy

负责定义具体输出策略，包括输出格式、输出路径和序列化方式。不同类型结果可以对应不同 OutputStrategy，例如 SARIF 报告、JSON 结果、调用图结果、污点路径结果或其他自定义结果。系统整体输出时，会统一遍历已经注册的 OutputStrategy，并调用对应输出方法，从 ResultManager 中读取相应类别的 Finding 或分析结果，最终生成目标格式的输出文件。通过这种方式，YASA-Engine 将结果生成、结果管理和结果输出解耦，使不同 Checker 的

输出既能够相互独立，又可以在需要进行组合。